

Retrieval Augmented Generation (RAG)

Why RAG Exists (Problems with Plain LLMs)

LLMs by themselves suffer from:

1. Limited knowledge access
 - Knowledge frozen at training time
 - No access to private or real-time data
2. Lack of accuracy and relevancy
 - Answers may be vague or outdated
 - Struggles with domain-specific queries
3. Hallucinations
 - Confidently generates incorrect information
 - No grounding in source data

RAG is designed to **reduce hallucinations** and **increase accuracy and relevance**.

What is RAG

Retrieval Augmented Generation (RAG) is an architecture that:

- Combines **information retrieval** with **text generation**
- Allows LLMs to answer using **external knowledge sources**
- Improves factual correctness by grounding responses in retrieved data

Definition

RAG is an architecture used to enhance LLM responses by retrieving relevant external data and injecting it into the prompt before generation.

RAG has **three core stages**:

1. Retrieval
2. Augmentation
3. Generation

Stage 1: Retrieval

Goal

Fetch the most relevant pieces of information from an external knowledge base.

Key Components

- User Query
- Embedding Model
- Vector Database
- Similarity Search (Cosine Similarity)
- Top-K Selection

Knowledge Ingestion (Offline / Preprocessing)

This happens **before users query the system**.

Steps

1. Data Source
 - PDFs
 - Documents
 - Policies
 - Internal knowledge base

2. Data Extraction

- Extract raw text/content from documents

3. Chunking

- Split large documents into smaller chunks
- Chunk size can be:
 - Word-level
 - Sentence-level
 - Paragraph-level
- Overlapping chunks are common to preserve context

4. Embedding

- Each chunk is converted into a vector using an embedding model
- Output: high-dimensional numerical vector

5. Storage

- Vectors are stored in a **Vector Database**

Vector Database

Stores:

- Embedding vectors
- Metadata (source, page number, section, etc.)

Common types:

- Vector DB (FAISS, Pinecone, Milvus, Weaviate)
- SQL DB with vector support
- Graph DB (for relationship-heavy data)

Query-Time Retrieval (Online)

1. User inputs query
2. Query is converted into an embedding
3. Similarity search is performed against stored vectors
4. Similarity metric:
 - Cosine Similarity (most common)

Cosine Similarity Intuition

- Measures angle between vectors
 - $\cos(0^\circ) = 1 \rightarrow$ identical meaning
 - $\cos(90^\circ) = 0 \rightarrow$ unrelated
 - Higher cosine score = more relevant
1. Retrieve **Top-K** most similar chunks

Stage 2: Augmentation

Goal

Create a **new enriched prompt** for the LLM.

What Happens

- Retrieved chunks are combined with the original user query
- This forms a **context-augmented prompt**

This step ensures:

- LLM answers only using retrieved knowledge
- Reduced hallucination

- Higher relevance

Stage 3: Generation

Goal

Generate the final answer using the LLM.

Process

- LLM receives:
 - User query
 - Retrieved context
- LLM generates a response grounded in provided data

